



Rapport de projet AutoEcole Manager

Application Symfony dockerisee pour la gestion d'une auto-ecole

Auteur : Idris ACHABOU

Etablissement : Universite Le Havre Normandie

Contexte : Projet Symfony - Licence Informatique

Perimetre : Symfony, Doctrine, PostgreSQL, Docker, reporting, qualite applicative

21 juin 2026

Table des matières

1	Introduction et contexte	1
2	Analyse de l'existant	1
2.1	Points positifs	1
2.2	Limites identifiées	2
3	Architecture cible	2
4	Conteneurisation et environnement d'exécution	3
4.1	Script multiplateforme	3
5	Modele de donnees	3
6	Regles metier	4
7	Fonctionnalites livrees	4
8	Qualite logicielle et maintenabilite	5
9	Validation et exploitation	5
10	Limites et perspectives	6
11	Conclusion	6
A	Commandes de reference	6
B	Structure finale du depot	7

CHAPITRE 1

Introduction et contexte

AutoEcole Manager est une application web de gestion d'auto-ecole concue autour du framework Symfony. L'application permet de suivre les candidats, les examens, les inscriptions, les paiements et les historiques administratifs. La version initiale couvrait deja les operations elementaires de gestion, mais elle restait limitee pour un usage demonstratif complet : environnement d'execution partiel, interface sommaire, migrations fragiles et logique metier dispersee dans plusieurs couches.

Le travail presente dans ce rapport consiste a transformer ce projet en une application plus robuste, plus facilement executable et plus proche d'un outil metier exploitable. L'objectif n'est pas seulement de dockeriser le depot ; il s'agit egalement de clarifier l'architecture, renforcer la logique de progression des examens, enrichir l'interface, ajouter des exports et preparer une base saine pour de futures extensions.

Objectifs principaux

- rendre l'application executable par Docker sans installation locale de PHP, Composer ou PostgreSQL ;
- fournir un script de lancement multiplateforme capable de detecter l'environnement ;
- nettoyer la configuration, les migrations et les secrets de developpement ;
- enrichir l'application avec des fonctions de pilotage, de filtrage, d'export et d'audit ;
- documenter le projet de maniere professionnelle et reproductible.

CHAPITRE 2

Analyse de l'existant

Le projet initial etait organise comme une application Symfony classique avec des entites Doctrine, des controleurs CRUD et des templates Twig. Trois concepts principaux etaient deja presents : les candidats, les examens et les inscriptions de candidats aux examens.

2.1 Points positifs

- l'utilisation de Symfony fournit une base solide pour le routage, les formulaires, la validation et l'injection de dependances ;
- Doctrine structure la persistance autour d'entites explicites ;
- Dompdf etait deja present pour la generation PDF ;
- des tests metier existaient autour de la logique de paiement et de progression ;
- la separation minimale entre controleurs, entites, formulaires et repositories etait deja amorcee.

2.2 Limites identifiées

Risques techniques de la version initiale

- la configuration de base contenait une URL PostgreSQL locale et personnelle;
- les migrations comportaient des traces d'un autre domaine métier et n'étaient pas propres pour une base vierge;
- l'environnement Docker était incomplet pour lancer l'application seule;
- l'interface reposait sur un CDN externe et restait peu exploitable;
- les listes ne proposaient pas de filtrage avancé, de KPI ou de lecture décisionnelle;
- certaines routes étaient sémantiquement peu lisibles;
- la logique métier devait être davantage centralisée.

CHAPITRE 3

Architecture cible

L'architecture cible repose sur une séparation claire entre couches. Les contrôleurs traitent les requêtes HTTP, les repositories portent les requêtes de consultation, les entités décrivent le modèle persistant et le service AutoEcoleManager concentre les règles métier transverses.

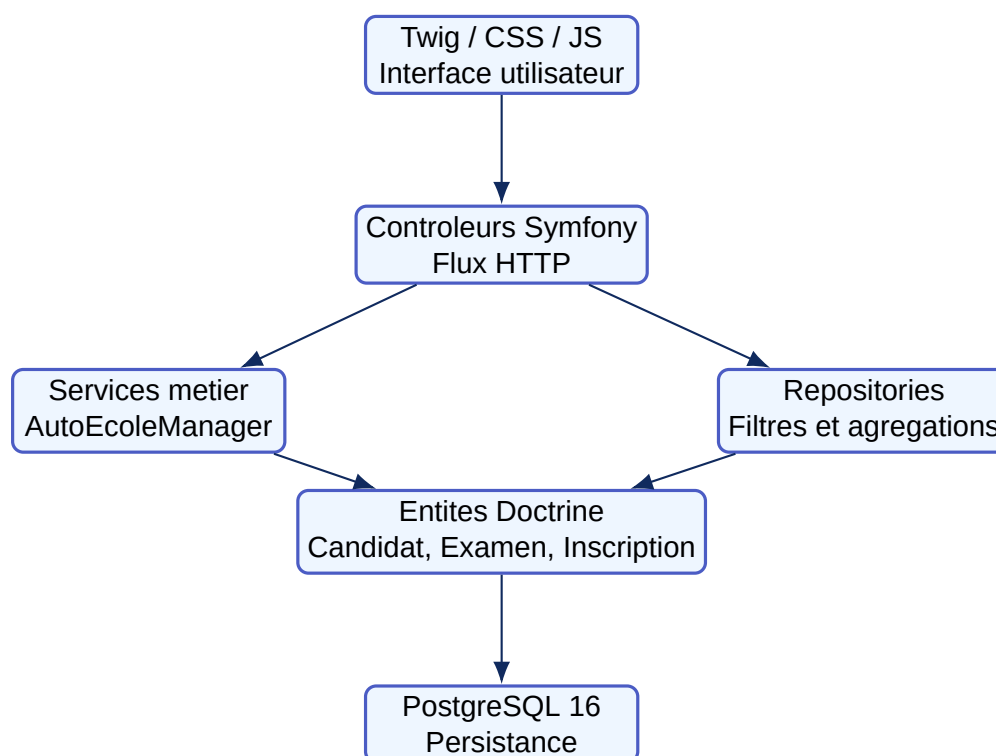


Figure 1 – Architecture logique de l'application enrichie.

CHAPITRE 4

Conteneurisation et environnement d'exécution

La dockerisation s'appuie sur quatre services : l'application Symfony, PostgreSQL, Adminer et Mailpit. Le conteneur applicatif installe les extensions PHP nécessaires, active Apache avec reécriture d'URL, installe Composer et applique les migrations au démarrage.

Service	Image / role	Utilite
app	PHP 8.3 Apache	Execute Symfony, installe les dependances Composer, applique les migrations et expose le port 8080.
database adminer	PostgreSQL 16 Alpine Adminer	Stocke les candidats, examens et inscriptions. Permet d'inspecter la base depuis un navigateur.
mailpit	Mailpit	Prepare les evolutions de notification email sans serveur SMTP externe.

Listing 1 – Lancement Docker principal

```
docker compose up --build
```

4.1 Script multiplateforme

Un script Python, `scripts/bootstrap_environment.py`, detecte le systeme d'exploitation et verifie la presence de Docker ainsi que du plugin Docker Compose. Selon le systeme, il propose une commande adaptee : `winget` sous Windows, `brew` sous macOS, ou le gestionnaire de paquets disponible sous Linux.

Choix de conception

Le script reste volontairement prudent. Il n'installe pas Docker sans confirmation explicite. Cette approche evite d'executer des commandes systeme invasives sans accord de l'utilisateur, tout en gardant un flux simple pour un poste neuf.

CHAPITRE 5

Modele de donnees

Le modele de donnees a ete nettoye dans une migration initiale unique afin de rendre la creation de base reproductible. Les anciennes migrations contenant des elements etrangers au domaine de l'auto-ecole ont ete remplacees par un schema coherent.

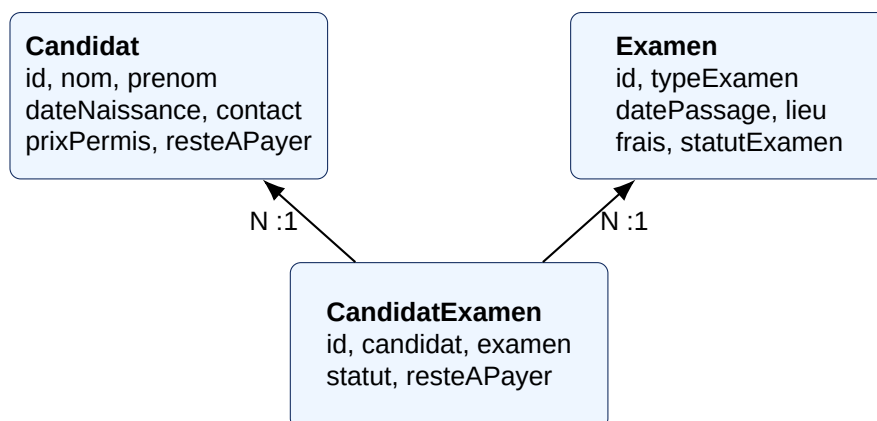


Figure 2 – Relations principales du modèle Doctrine.

CHAPITRE 6

Regles metier

La règle de progression des examens est centralisée. Un candidat peut passer le code en première étape. Le créneau devient accessible après réussite du code. La conduite devient accessible après réussite du créneau.

Listing 2 – Principe de progression dans AutoEcoleManager

```
Code réussi -> Créneau autorisé -> Conduite autorisée
```

Le paiement est également normalisé. Un montant positif réduit le reste à payer. Le solde ne peut jamais devenir négatif. Lorsque le reste atteint zéro, l'objet modifie automatiquement le statut financier concerné.

CHAPITRE 7

Fonctionnalités livrées

No	Fonctionnalité	Description
1	Tableau de bord	Vue centrale avec KPI candidats, paiements, examens et alertes.
2	KPI financiers	Total des contrats, encaisse, reste permis, reste examens.
3	Alertes paiement	Mise en avant des dossiers avec solde restant significatif.
4	Alertes résultat	Détection des examens passés sans résultat saisi.
5	Sessions à venir	Liste synthétique des prochaines dates d'examen.
6	Recherche candidats	Recherche par nom, prénom, email ou téléphone.
7	Filtres candidats	Filtrage par statut de paiement et groupe sanguin.
8	Tri candidats	Tri par nom, âge, prix ou reste à payer.
9	Export candidats CSV	Extraction tableur pour reporting administratif.
10	Export candidats JSON	Extraction structurée pour réutilisation applicative.

No	Fonctionnalite	Description
11	Dossier candidat	Vue detaillee avec progression, paiements et alertes.
12	Timeline pedagogique	Suivi code, creneau, conduite pour chaque candidat.
13	Paiement permis	Paiement partiel directement depuis le dossier.
14	Filtres examens	Filtrage des sessions par type, lieu et periode.
15	Export examens CSV	Export des sessions et volumes d'inscription.
16	Detail examen	Consultation des candidats rattaches a une session.
17	Controle progression	Refus d'inscription si l'etape precedente n'est pas validee.
18	Filtres inscriptions	Recherche par candidat, statut, type et paiement.
19	Mise a jour statut	Modification rapide : inscrit, paye, reussi, echoue.
20	Paiement examen	Paiement partiel des frais d'examen.
21	Historique filtre	Consultation administrative par nom, prenom et dates.
22	Export historique CSV	Extraction tabulaire des historiques.
23	Export historique JSON	Extraction structuree pour integration.
24	Export historique PDF	Edition PDF via Dompdf.
25	Healthcheck	Endpoint JSON /healthz pour supervision.
26	Seed demo	Commande idempotente app:seed-demo.
27	Audit metier	Commande app:quality-audit.
28	Interface locale	CSS local, responsive, sans CDN obligatoire.
29	Adminer	Inspection DB via conteneur dedie.
30	Mailpit	Preparation des notifications sans SMTP externe.

CHAPITRE 8

Qualite logicielle et maintenabilite

Le projet a ete retravaille pour limiter les dependances implicites. Les secrets personnels ont ete retires de la configuration versionnee, les migrations ont ete rationalisees et les styles sont servis localement. Les classes metier exposent des methodes calculees explicites : age, solde, progression paiement, progression examen et prochain examen.

Points de maintenabilite

- les controleurs restent responsables du flux HTTP ;
- les repositories concentrent les requetes de filtrage ;
- le service metier centralise les regles de progression ;
- les commandes CLI facilitent le seed et l'audit ;
- Docker fournit un environnement stable et partageable.

CHAPITRE 9

Validation et exploitation

Une commande de controle locale est fournie :

```
scripts/quality.sh
```

Dans un environnement Docker complet, cette commande peut etre completee par les lints Symfony, les tests PHPUnit et l'audit metier :

```
docker compose exec app php bin/console lint:twig templates
docker compose exec app php bin/console lint:yml config
docker compose exec app php bin/phpunit
docker compose exec app php bin/console app:quality-audit
```

CHAPITRE 10

Limites et perspectives

Cette version constitue une base solide pour une demonstration et une extension progressive. Elle ne couvre pas encore la gestion des moniteurs, les lecons de conduite, les contrats PDF signes, les pieces jointes, l'authentification par role ou l'envoi effectif de notifications. Ces evolutions sont naturelles et peuvent etre ajoutes sans remettre en cause l'architecture existante.

Extensions prioritaires

- authentification avec roles administrateur, secretaire et moniteur;
- planning des heures de conduite;
- generation de factures PDF;
- depot et validation des pieces administratives;
- notifications email via Messenger et Mailpit en developpement;
- API REST pour une future interface mobile.

CHAPITRE 11

Conclusion

Le projet AutoEcole Manager a ete transforme en application Symfony dockerisee, mieux structuree et plus riche fonctionnellement. La mise en place de Docker rend l'execution reproductible, PostgreSQL remplace les dependances locales implicites, et l'interface fournit une lecture metier plus claire. Les ajouts en matiere de reporting, d'exports, d'audit et de controle de progression permettent de passer d'un CRUD simple a une application de gestion plus coherente et extensible.

ANNEXE A

Commandes de reference

```
# Lancement direct
docker compose up --build

# Lancement avec detection OS
python scripts/bootstrap_environment.py

# Reinitialisation complete
python scripts/bootstrap_environment.py --reset
```


B. Structure finale du depot

```
# Seed de demonstration
docker compose exec app php bin/console app:seed-demo

# Audit metier
docker compose exec app php bin/console app:quality-audit
```

ANNEXE B

Structure finale du depot

```
autoecole/|—
Dockerfile|—
compose.yaml|—
docker/|—
scripts/|—
src/|
  |— Command/|
  |— Controller/|
  |— Entity/|
  |— Form/|
  |— Repository/|
  |— Service/|—
templates/|—
public/|—
migrations/|—
README.md
```